Estructuras de Datos

FICHA TÉCNICA DE LA UNIDAD

Unidad trabajo	UT3. Estructuras de datos
Resultados de Aprendizaje	RA4. Implementa estructuras de datos lineales y no lineales, evaluando su eficiencia y aplicando algoritmos de búsqueda y ordenación.
Criterios de Evaluación	CE4.a: Gestionar arrays unidimensionales y multidimensionales de forma estática y dinámica. CE4.b: Modelar e implementar estructuras lineales complejas (listas enlazadas, pilas y colas). CE4.c: Conceptualizar y programar recorridos de estructuras jerárquicas no lineales (árboles y grafos). CE4.d: Aplicar algoritmos de ordenación y búsqueda optimizada. CE4.e: Evaluar la complejidad y eficiencia temporal.
Tiempo	15 horas

3.1. Introducción

En muchos lenguajes de programación tradicionales se implementan los arrays (o arreglos), los cuales son estructuras de datos que almacenan elementos homogéneos (del mismo tipo, ya sea numérico o alfanumérico) reconocidos bajo un nombre común. Sin embargo, los arrays tradicionales presentan dos grandes limitaciones:

---

1. Homogéneos: Solo permiten guardar datos del mismo tipo.
2. Estáticos: Al declararse se debe indicar su tamaño, y la longitud del array no se puede modificar durante la ejecución del programa.

En Python no existen los arrays tradicionales. En su lugar, el lenguaje proporciona varios tipos de datos nativos dinámicos y flexibles que permiten almacenar colecciones de información de forma eficiente, como las listas.

3.2. El Tipo de Dato LISTA (list)

Las listas son uno de los tipos de datos más utilizados en programación. Permiten almacenar un conjunto de datos que pueden repetirse y que pueden ser de distintos tipos (heterogéneas). Además, es una estructura dinámica, lo que significa que se pueden añadir o eliminar elementos modificando su tamaño en cualquier momento de la ejecución.

3.2.1. Construcción y Declaración de Listas

Para crear una lista en Python se utilizan los caracteres de corchetes [y].

```
# Sintaxis general
lista = [valor1, valor2, ...]
# Ejemplos de declaración
lista_vacia = [] # Crea una lista vacía
enteros = [1, 4, 56, 78, 100] # Lista de números enteros
decimales = [1.4, 4, 56.5, 78, 100.3] # Lista de números
decimales
cadenas = ['hola', 'adios', 'buena suerte'] # Lista de cadenas de
caracteres
mixta = ["a", 1, True] # Lista con distintos tipos de datos
inicializada = [0, 0, 0, 0, 0, 0] # Lista de 6 elementos
inicializados a 0
```

3.2.2. Indexación y Acceso

Cada uno de los elementos de una lista está indexado por una posición. Las posiciones comienzan en 0 y terminan en el número de elementos menos 1.

También se pueden utilizar índices negativos para acceder desde el final de la lista hacia atrás (donde -1 es el último elemento).

```
loteria = [3, 32, 12, 21, 16, 33]
print(loteria[0]) # Muestra 3
print(loteria[5]) # Muestra 33
print(loteria[-1]) # Muestra 33 (índice negativo)
# ERROR DE ÍNDICE
```

```
print(loteria[6]) # Produce un error IndexError: list index out of
range
```

Si intentamos asignar directamente un valor a una posición que no existe para expandir la lista, Python lanzará un error:

```
valores = [1, 2, 3, 4, 5]
valores[5] = 6 # ERROR: La posición 5 no existe en una lista de
longitud 5
```

3.2.3. Operaciones Básicas con Listas

Las listas son secuencias y permiten realizar operaciones directas con operadores integrados:

- Recorrido simple (for):
-

```
lista = [1, 2, 3, 4, 5, 6]
for num in lista:
print(num, end="") # Resultado: 123456
```

Recorrido múltiple (zip): Permite recorrer más de una lista simultáneamente en el mismo ciclo.

```
lista = [1, 2, 3, 4, 5, 6]
lista2 = ["a", "b", "c", "d", "e"]
for num, letra in zip(lista, lista2):
print(num, letra)
# Resultado: 1 a / 2 b / 3 c / 4 d / 5 e
```

Operadores de pertenencia (in y not in): Comprueban si un elemento pertenece o no a la lista.

```
2 in lista # Devuelve True
8 not in lista # Devuelve True
```

Concatenación (+): Permite unir dos listas en una nueva.

```
lista + [7, 8, 9] # Resultado: [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Repetición (*): Repite los elementos de la lista un número de veces.

```
lista * 2 # Resultado: [1, 2, 3, 4, 5, 6, 1, 2, 3, 4, 5, 6]
```

Slicing / Rebanadas ([inicio:fin:paso]): Obtiene una subsecuencia de la lista.

```
lista = [1, 2, 3, 4, 5, 6]
print(lista[2:4]) # [3, 4] (desde posición 2 hasta 4 sin incluir)
print(lista[1:4:2]) # [2, 4] (desde 1 hasta 4, de 2 en 2)
print(lista[:5]) # [1, 2, 3, 4, 5] (desde el inicio hasta la posición
5)
print(lista[5:]) # [6] (desde la posición 5 hasta el final)
print(lista[::-1]) # [6, 5, 4, 3, 2, 1] (invierte la lista usando
paso negativo)
```

3.2.4. Funciones Globales Predefinidas

Funciones nativas de Python que aceptan una lista como argumento:

```
lista1 = [20, 40, 10, 40, 50]
len(lista1) # 5 -> Devuelve la longitud/número de elementos
max(lista1) # 50 -> Devuelve el valor máximo
min(lista1) # 10 -> Devuelve el valor mínimo
sum(lista1) # 160 -> Devuelve la suma de todos los elementos
sorted(lista1) # [10, 20, 40, 40, 50] -> Devuelve una NUEVA lista
ordenada sin modificar la original
sorted(lista1, reverse=True) # [50, 40, 40, 20, 10] -> Nueva lista
ordenada de mayor a menor
```

3.2.5. Concepto de Mutabilidad y Copia de Listas

Las listas son objetos pertenecientes a una clase y son mutables, lo que significa que sus elementos pueden modificarse directamente en memoria.

```
lista1 = [1, 2, 3]
lista1[2] = 4 # Modifica el elemento en la posición 2
print(lista1) # [1, 2, 4]
del lista1[2] # Borra el elemento en la posición 2 usando la
instrucción 'del'
print(lista1) # [1, 2]
```

¿Cómo se copian las listas correctamente?

No se puede utilizar el operador de asignación (=) para copiar una lista. Este operador no crea una lista nueva, sino que crea una referencia adicional (un alias) apuntando al mismo objeto en memoria.

```
# Intento incorrecto de copia (Asignación de referencia)
lista1 = [1, 2, 3]
lista2 = lista1
lista1[1] = 10
print(lista2) # [1, 10, 3] -> ¡La lista 2 también cambió!
# Forma correcta de clonar/copiar una lista (Slicing completo)
lista1 = [1, 2, 3]
lista2 = lista1[:] # Se extrae una rebanada completa generando un
nuevo objeto
lista1[1] = 10
print(lista2) # [1, 2, 3] -> La lista original cambia, pero la
copia se mantiene intacta
```

3.2.6. Métodos Propios del Objeto Lista

Los métodos se invocan usando la nomenclatura lista.metodo() y modifican directamente el contenido de la lista (in-place).

Métodos de Inserción / Adición

- append(valor): Añade un único elemento al final de la lista.
-

```
valores = [1, 2, 3, 4, 5]
valores.append(6)
```

```
print(valores) # [1, 2, 3, 4, 5, 6]
```

extend(lista_a_unir): Une una lista al final de otra (concatenación in-place).

```
lista = [1, 2, 3, 4]
lista.extend([5, 6])
print(lista) # [1, 2, 3, 4, 5, 6]
```

insert(posicion, valor): Inserta un elemento en una posición concreta, desplazando los siguientes.

```
valores = [1, 2, 3, 4, 5]
valores.insert(1, 12) # Inserta el 12 en la posición 1
print(valores) # [1, 12, 2, 3, 4, 5]
```

Métodos de Eliminación

- pop([indice]): Elimina un elemento de la lista y lo devuelve. Si no se indica parámetro, saca el último elemento.

```
valores = [1, 2, 3, 4, 5]
valores.pop() # Devuelve 5, la lista queda [1, 2, 3, 4]
valores.pop(0) # Devuelve 1 (saca la posición 0), la lista queda [2,
3, 4]
```

remove(valor): Busca y elimina el primer elemento cuyo valor coincida con el argumento dado.

```
valores = [1, 2, 3, 5]
valores.remove(3) # Quita el número 3
print(valores) # [1, 2, 5]
```

Métodos de Ordenación

- reverse(): Invierte el orden de los elementos de la lista en el lugar.
-

```
lista = [1, 2, 4, 5]
lista.reverse()
print(lista) # [5, 4, 2, 1]
```

sort([reverse=True]): Ordena los elementos de la lista de forma definitiva.

```
lista = [5, 4, 2, 1]
lista.sort() # Ordena de menor a mayor
print(lista) # [1, 2, 4, 5]
lista.sort(reverse=True) # Ordena de mayor a menor
print(lista) # [5, 4, 2, 1]
# Ordenación de cadenas de texto (Orden alfabético/ASCII: Mayúsculas
primero)
texto = ["hola", "que", "tal", "Hola", "Que", "Tal"]
texto.sort()
print(texto) # ['Hola', 'Que', 'Tal', 'hola', 'que', 'tal']
```

Métodos de Búsqueda

- count(valor): Devuelve el número de veces que aparece un valor en la lista.
-

```
valores = [5, 4, 2, 1, 5]
print(valores.count(5)) # Devuelve 2
```

index(valor[, inicio, fin]): Devuelve la posición de la primera aparición de un valor. Se puede acotar el rango de búsqueda con los parámetros opcionales inicio y fin. Si el elemento no existe, lanza un error/excepción.

```
valores = [5, 4, 2, 1, 5]
valores.index(5) # Devuelve 0 (primera aparición)
valores.index(5, 1) # Devuelve 4 (busca a partir del índice 1)
valores.index(5, 1, 4) # ERROR ValueError: 5 is not in list
```

3.2.7. Listas Multidimensionales

Dado que las listas aceptan cualquier tipo de datos, es perfectamente posible anidar listas dentro de listas, creando matrices o tablas bidimensionales.

```
# Definición de una matriz de 3x3
tabla = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
# Acceso (Fila, Columna)
print(tabla[1][1]) # Accede a la fila 1, columna 1 -> Devuelve
5
# Recorrido anidado
for fila in tabla:
for elem in fila:
print(elem, end="")
print() # Salto de línea por cada fila
# Resultado visual:
# 123
# 456
# 789
```

3.3. El Tipo de Dato TUPLA (tuple)

Las tuplas sirven exactamente para el mismo propósito conceptual que las listas (guardar colecciones ordenadas de datos repetibles y heterogéneos), con la diferencia crucial de que son inmutables. Una vez creada una tupla, no se puede modificar su tamaño ni cambiar sus valores.

3.3.1. Construcción de Tuplas

Para crear una tupla se utilizan los caracteres de paréntesis (y).

```
tupla1 = () # Tupla vacía
tupla2 = ("a", 1, True) # Tupla con tres elementos heterogéneos
```

3.3.2. Operaciones Permitidas e Inmutabilidad

Se pueden realizar operaciones de lectura idénticas a las listas:

- Recorrido con bucles for.
- Operadores de pertenencia: in y not in.

- Concatenación (+) y Repetición (*) generando una nueva tupla resultante.
- Indexación y Slicing.
- Funciones globales: len(), max(), min(), sum(), sorted().

Demostración de Inmutabilidad:

```
tupla = (1, 2, 3)
tupla[1] = 5 # ERROR: TypeError: 'tuple' object does not support item
assignment
```

3.3.3. Métodos de Búsqueda de las Tuplas

Al no ser modificables, solo poseen los métodos de búsqueda:

```
tupla = (1, 2, 3, 4, 1, 2, 3)
print(tupla.count(1)) # Devuelve 2
print(tupla.index(2)) # Devuelve 1 (primera aparición)
print(tupla.index(2, 2)) # Devuelve 5 (busca a partir de la posición
2)
```

3.4. El Tipo de Dato MAPA / DICCIONARIO (dict)

Los diccionarios son estructuras de datos que permiten almacenar parejas de clave-valor. A los valores se accede directamente por medio de su clave (usualmente cadenas de texto) en lugar de un índice numérico. Son estructuras mutables y sus campos no tienen un orden asignado internamente.

3.4.1. Definición y Construcción de Diccionarios

Se definen utilizando llaves {} y declarando pares clave: valor.

```
# Creación directa con elementos
diccionario = {'one': 1, 'two': 2, 'three': 3}
```

```
# Construcción progresiva a partir de un diccionario vacío
dict1 = {}
dict1["one"] = 1
dict1["two"] = 2
dict1["three"] = 3
```

3.4.2. Operaciones Básicas

```
a = {'one': 1, 'two': 2, 'three': 3}
# len(): Devuelve el número de elementos (pares clave-valor)
print(len(a)) # Devuelve 3
# Indexación para lectura o modificación
print(a["one"]) # Devuelve 1
a["one"] += 1 # Modifica el valor asociado a la clave
# Si se intenta acceder a una clave que no existe, lanza un error
KeyError
# del: Elimina un elemento por su clave
del a["one"] # Elimina la clave "one" y su valor
# Operadores de pertenencia: Comprueban si una CLAVE existe en el
diccionario
"two" in a # Devuelve True
```

3.4.3. Mutabilidad y Copia de Diccionarios

Al igual que las listas, si asignamos un diccionario a otra variable usando =, ambas apuntarán al mismo objeto. Para duplicar un diccionario de forma independiente se debe utilizar el método .copy().

```
# Copia incorrecta por referencia
a = {'one': 1, 'two': 2}
b = a
del a["one"]
print(b) # {'two': 2} -> b también se ve afectado
# Copia correcta usando .copy()
a = {'one': 1, 'two': 2, 'three': 3}
b = a.copy()
a["one"] = 1000
print(b) # {'three': 3, 'one': 1, 'two': 2} -> El diccionario 'b'
se mantiene intacto
```

3.4.4. Métodos Principales de los Diccionarios

- Eliminación:
    - clear(): Vacía por completo todos los elementos del diccionario.
-

```
dict1 = {'one': 1, 'two': 2}
dict1.clear() # La variable queda como {}
```

Agregado y Actualización:

- update(otro_dict): Agrega elementos o actualiza los valores existentes a partir de otro diccionario.

```
dict1 = {'one': 1, 'two': 2}
dict2 = {'three': 3, 'four': 4}
dict1.update(dict2) # dict1 ahora contiene las claves de ambos
```

Retorno Seguro y Extracción:

- get(clave[, valor_por_defecto]): Devuelve el valor de una clave. Si la clave no existe, no lanza un error, sino que devuelve None o el valor por defecto indicado.

```
dict1 = {'one': 1, 'two': 2}
dict1.get("one") # Devuelve 1
dict1.get("four") # Devuelve None
dict1.get("four", "no existe") # Devuelve la cadena 'no existe'
```

pop(clave[, valor_por_defecto]): Devuelve el valor asociado a la clave y elimina el elemento. Si no existe la clave, lanza KeyError a menos que se indique un valor por defecto.

```
dict1.pop("one") # Devuelve 1 y lo borra del diccionario
dict1.pop("four", "no existe") # Devuelve 'no existe' de manera
segura
```

3.4.5. Recorrido de Diccionarios

Existen tres métodos fundamentales para iterar sobre un diccionario:

```
dict1 = {'one': 1, 'two': 2, 'three': 3}
# 1. .keys(): Recorrer solo las claves
for clave in dict1.keys():
print(clave) # Imprime: one, two, three
# 2. .values(): Recorrer solo los valores
for valor in dict1.values():
print(valor) # Imprime: 1, 2, 3
# 3. .items(): Recorrer clave y valor simultáneamente
for clave, valor in dict1.items():
print(clave, "->", valor)
```

3.5. Cadenas de Caracteres (str) y sus Operaciones

Las cadenas de caracteres son sucesiones de caracteres alfanuméricos encerrados entre comillas. En Python se comportan en gran medida como secuencias inmutables (similares a tuplas de caracteres).

Al igual que las listas, la primera posición de una cadena es la 0 y la última es su longitud menos 1.

3.5.1. Funciones y Operaciones Básicas

```
# Longitud (len): Retorna el conteo total de caracteres.
cadena = "Hola mundo"
print(len(cadena)) # Resultado: 10 (incluye el espacio)
```

```
# Acceso por Posición:
print(cadena[2]) # Resultado: 'l'
```

```
# Concatenación (+ y +=): Une cadenas de caracteres de forma
secuencial.
cadena = "Hola" + " " + "mundo" # Resultado: 'Hola mundo'
```

```
# Conversión a cadena (str()): Es obligatorio convertir datos
numéricos a texto si se desea concatenarlos con strings.
precio = 34
cadena = "El precio del producto serían " + str(precio) + " euros" #
Correcto
```

3.5.2. Métodos Principales de Cadenas

- Modificación de mayúsculas/minúsculas:
  - upper(): Convierte todo a mayúsculas.
  - lower(): Convierte todo a minúsculas.
  - capitalize(): Pone la primera letra en mayúscula y el resto en minúsculas.
- Búsqueda:
  - find(subcadena): Busca la posición de inicio de una subcadena. Si la encuentra, devuelve el índice; si no, devuelve -1.
-

```
cadena = "Hola mundo"
print(cadena.find("mundo")) # 5
print(cadena.find("mondo")) # -1
print(cadena.find("o")) # 1 (posición de la primera 'o')
```

Reemplazo:

- replace(antiguo, nuevo): Reemplaza las apariciones de una subcadena por otra.

```
cadena = "Hola mundo"
cadena2 = cadena.replace("Hola", "Ostras") # "Ostras mundo"
```

Troceado y Segmentación:

- Por Slicing ([inicio:fin]): Corta seleccionando una parte basada en posiciones.

```
cadena = "Hola mundo"
cadena2 = cadena[1:7]
print(cadena2) # "ola mu" (desde índice 1 al 6)
```

Por método split(): Parte la cadena en trozos basándose en un delimitador (por defecto, espacios) y los devuelve en una lista.

```
cadena = "Hola mundo"
print(cadena.split()) # ['Hola', 'mundo']
```

3.6. Manejo de Errores y Excepciones

3.6.1. Errores Sintácticos vs. Excepciones

- Errores Sintácticos (SyntaxError): Se producen cuando el código está mal escrito formalmente y el intérprete no puede iniciar el programa.
-

```
while True print('Hello world') # ERROR: SyntaxError: invalid syntax
(faltan los ':')
```

Excepciones / Errores de ejecución: Errores que ocurren mientras el programa ya se está ejecutando. Si no se manejan, detienen el programa de inmediato.

```
4 / 0 # ZeroDivisionError: division by zero
a + 4 # NameError: name 'a' is not defined
"2" + 2 # TypeError: Can't convert 'int' object to str implicitly
```

3.6.2. El Bloque try - except - else - finally

Para capturar y manejar estas situaciones de forma controlada se utiliza la estructura estructurada de bloques:

1. try: Aloja el bloque de código que se quiere intentar ejecutar y que podría fallar.
2. except: Captura la excepción específica. Si ocurre el error dentro del try, se interrumpe y salta a este bloque.
3. else: Bloque opcional. Se ejecuta únicamente si el bloque try se completó con éxito.
4. finally: Bloque opcional. Se ejecuta siempre, independientemente de si ocurrió un error o no.

```
while True:
try:
x = int(input("Introduce un número:"))
break
except ValueError:
print("Debes introducir un número")
# Control de múltiples excepciones
try:
print(10 / int(cad))
except ValueError:
print("No se puede convertir a entero")
except ZeroDivisionError:
print("No se puede dividir por cero")
except:
print("Otro error no especificado")
```

3.7. Gestión de Archivos / Ficheros Nativos

3.7.1. La Función open() y Modos de Acceso

La función open() se utiliza para abrir un archivo, requiriendo normalmente la ruta del archivo y el modo de acceso. Todo archivo abierto manualmente debe cerrarse explícitamente con .close().

```
f = open("ejemplo.txt", "w") # Abre un fichero en modo escritura
print(type(f)) # Muestra <class '_io.TextIOWrapper'>
f.close() # Cierra el archivo
```

Tabla Resumen de Modos de Acceso:

Modo	Nombre	Descripción / Comportamiento	¿Crea si no existe?	¿Borra previo contenido?	Posición Puntero
r	Leer	Solo lectura. Lanza error si el archivo no existe.	No	No	Al inicio
rb	Lectura Binaria	Solo lectura de archivos en formato binario.	No	No	Al inicio
w	Escribir	Solo escritura. Sobrescribe por completo si existe, lo crea si no.	Sí	Sí	Al inicio
wb	Escritura Binaria	Solo escritura en formato binario. Sobrescribe.	Sí	Sí	Al inicio
a	Anexar / Apéndice	Modo añadir contenido. Escribe al final del texto sin borrar.	Sí	No	Al final
r+	Mixto estándar	Lectura y escritura combinadas.	No	No	Al inicio
w+	Mixto Escritura	Escritura y lectura. Sobrescribe el contenido previo.	Sí	Sí	Al inicio
a+	Mixto Anexar	Añadido y lectura. Crea el archivo si no existe.	Sí	No	Al final

3.7.2. Codificación (encoding) y Propiedades del Objeto Fichero

Al abrir archivos de texto, es recomendable usar el parámetro encoding="utf-8" para garantizar la correcta interpretación de caracteres especiales.

```
import locale
print(locale.getpreferredencoding()) # Retorna la codificación por
defecto del sistema
```

Propiedades de inspección básicas:

- .closed: Retorna True si el archivo ha sido cerrado con éxito.
- .mode: Retorna la cadena del modo de apertura (ej: "r").
- .name: Retorna la ruta/nombre del archivo.
- .encoding: Retorna la codificación utilizada.

3.7.3. Métodos de Lectura y Control del Cursor

- read([n]): Lee el contenido completo o los siguientes n caracteres si se especifica.
- readline(): Lee una única línea completa de texto (hasta encontrar \n).
- readlines(): Lee todas las líneas del archivo y las devuelve en una lista de cadenas.
- tell(): Informa la posición numérica exacta en bytes en la que se encuentra el cursor.
- seek(posicion): Desplaza el cursor de forma manual a la posición especificada (ej: seek(0) vuelve al inicio).

```
f = open("ejemplo.txt", "r")
print(f.read(4)) # Imprime los primeros 4 caracteres
print(f.tell()) # Muestra la posición del cursor
f.seek(0) # Reubica el puntero al inicio absoluto
lineas = f.readlines()
f.close()
```

3.7.4. Métodos de Escritura

- write(cadena): Escribe una cadena de texto plana. No introduce saltos de línea automáticos; requiere añadir \n.
- writelines(lista_de_cadenas): Toma una lista de textos e inyecta todos los elementos seguidos en el archivo.
- Redirección de print(): Es posible usar print(..., file=objeto_fichero).

```
f = open("ejemplo3.txt", "w")
f.write("Prueba 1\n")
```

```
print("Prueba 2\n", file=f)
f.writelines(["Prueba 3", "Prueba 4"])
f.close()
```

3.7.5. Buenas Prácticas: La Sentencia with open

La forma estándar y recomendada para manipular archivos en Python es utilizar el gestor de contexto mediante la cláusula with open(...) as alias:.

```
try:
with open("archivo_datos.txt", "r", encoding="utf-8") as f:
for linea in f: # Recorrido directo y eficiente línea a línea sin
saturar la RAM
print(linea)
except FileNotFoundError:
print("Error: El archivo no se encontró.")
except Exception as e:
print(f"Ocurrió un error inesperado: {e}")
```

3.8. Archivos Estructurados: Formato CSV

El formato CSV (Comma-Separated Values) representa datos tabulares sencillos en texto plano. Para gestionarlos correctamente, Python incluye de manera nativa el módulo csv.

3.8.1. Lectura de Archivos CSV

```
import csv
with open("ejemplo1.csv", "r", encoding="utf-8") as fichero:
contenido = csv.reader(fichero) # Genera un objeto iterador
# Opción A: Convertir todo directamente a una lista de listas
datos = list(contenido)
print(datos[0][0]) # Acceso directo a Fila 0, Columna 0
```

Lectura avanzada con parámetros adicionales (quotechar):

```
with open("ejemplo2.csv", "r") as fichero:
contenido = csv.reader(fichero, quotechar='"')
```

```
for row in contenido:
print(f"Fila {contenido.line_num}: {row}")
```

3.8.2. Escritura de Archivos CSV

- writerow(lista): Escribe una única fila a partir de una lista de elementos.
- writerows(lista_de_listas): Recibe una colección de filas y las escribe todas consecutivamente.

```
import csv
with open("ejemplo3.csv", "w", newline="", encoding="utf-8") as
fichero:
contenido = csv.writer(fichero)
contenido.writerow(['4/5/2015 13:34', 'Apples', '73'])
contenido.writerows([
['4/5/2015 3:41', 'Cherries', '85'],
['4/6/2015 12:46', 'Pears', '14']
])
```

3.9. Archivos Estructurados: Formato JSON

El formato JSON (JavaScript Object Notation) es un estándar abierto muy extendido para el intercambio de datos estructurados. En Python se gestiona importando el módulo json.

Mapea de forma directa sus tipos nativos: los objetos JSON se transforman en diccionarios (dict) y los arreglos JSON en listas (list).

3.9.1. Lectura de Estructuras JSON

- json.loads(cadena_texto): Decodifica e interpreta una estructura JSON contenida dentro de una variable de texto (String).
- json.load(objeto_fichero): Decodifica e interpreta los datos directamente leyendo desde un archivo físico en disco.

```
import json
# Caso 1: Desde una cadena de texto (String)
datos_json = '{"nombre":"carlos", "edad":23}'
```

```
datos = json.loads(datos_json)
print(type(datos)) # Muestra <class 'dict'>
print(datos["nombre"]) # Acceso por clave -> 'carlos'
# Caso 2: Desde un archivo físico .json en disco
with open("ejemplo1.json", "r", encoding="utf-8") as fichero:
datos_archivo = json.load(fichero)
print(type(datos_archivo)) # <class 'dict'>
```

3.9.2. Escritura de Estructuras JSON

Para exportar un diccionario o estructura de Python a un archivo en formato JSON se utiliza la función json.dump(estructura, objeto_fichero).

```
import json
datos_gato = {
'isCat': True,
'miceCaught': 0,
'name': 'Zophie',
'felineIQ': None
}
with open("ejemplo2.json", "w", encoding="utf-8") as fichero:
json.dump(datos_gato, fichero)
```

Nota de Traducción de Tipos: Al realizarse la exportación, Python traduce automáticamente sus palabras clave a los equivalentes universales de JSON (por ejemplo, el booleano True pasa a ser true, y el objeto de ausencia de valor None se traduce a null).